

1. ¿Qué es JSX en React? ¿Cuáles son sus características y para qué sirve? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descrito, explicando el ejemplo.

JSX es una extensión de sintaxis para JavaScript que permite escribir marcado similar a HTML dentro de un archivo JavaScript.

Sirve para diseñar interfaces de usuario de forma visual, limpia y fácil de leer sin usar funciones repetitivas. Permite a los desarrolladores combinar la estructura visual de un componente y su lógica de programación en un solo lugar, facilitando su lectura y mantenimiento.

Como rasgos característicos, podemos marcar que:

- Al tener una sintaxis parecida a HTML mejora la legibilidad, comprensión y manejo del código
- Permite insertar y evaluar expresiones de JavaScript directamente dentro del marcado visual utilizando llaves.
- Sigue la arquitectura basada en componentes de React
- Los navegadores no entienden JSX directamente, por lo que debe **transpirarse/transpilarse** a código JavaScript estándar utilizando herramientas como Babel. (transpilar: Traducir un lenguaje de alto nivel a otro lenguaje de alto nivel).
- Es declarativo
- Un bloque de JSX siempre debe devolver un único elemento raíz que envuelva a los demás.
- Es obligatorio cerrar absolutamente todas las etiquetas, incluso las que están vacías.
- Los atributos de las etiquetas HTML se deben escribir utilizando la convención camelCase

```
function SaludoInteractivo() {  
  
  const nombreUsuario = "Estudiante de IS2";  
  
  // Retorno de estructura visual JSX  
  return (  
    // Devolver un solo elemento raíz (ETIQUETA VACÍA)  
    <>  
      /* JavaScript dentro de JSX con llaves {} */  
      /* Uso de camelCase para los atributos */  
      <h1 className="saludo">¡Hola, {nombreUsuario}!</h1>  
  
      <p>Bienvenido a la demostración de JSX.</p>  
  
        
    </>  
    // Cierre de etiqueta vacía  
  );  
}
```

2. ¿Qué son los componentes en React? ¿Cuáles son sus características y para qué sirve? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descrito, explicando el ejemplo.

Un componente de React es una función de JavaScript a la que puedes añadir marcado. O en otras palabras es una función de JS que devuelve código de marcado (JSX) para describir cómo debe visualizarse una sección específica de la interfaz en la pantalla. React toma este enfoque para poder construir una UI con elementos independientes y reutilizables.

En los componentes, la independencia y reutilización van de la mano; cada componente es libre y solo encapsula y modifica su propia estructura visual (podemos modificar un componente sin miedo a romper el resto), esto permite que usemos este componente cuantas veces queramos en distintas partes de nuestro código sin miedo a afectar el resto. Los componentes también pueden estar compuestos por otros componentes más pequeños. Siempre deben ser nombrados con una primera letra mayúscula.

Nos permiten organizar y simplificar el desarrollo de interfaces complejas, así como también nos aporta escalabilidad en nuestros proyectos. Facilitan el trabajo en equipo y la mantenibilidad.

```
test.py

// El componente debe empezar con mayúscula
function TarjetaProducto() {
  return (
    // Devuelve un único elemento raíz en sintaxis JSX
    <div className="tarjeta">
      <h2>Zapatillas Deportivas</h2>
      <p>Cómodas y ligeras para correr.</p>
      <button>Comprar ahora</button>
    </div>
  );
}

// Componente que compone la interfaz
export default function Catalogo() {
  return (
    <section>
      <h1>Catálogo de Productos</h1>

      { /* Reutilización */ }
      { /* Se inserta como si fuera una etiqueta HTML auto-cerrada */ }
      <div className="grilla-productos">
        <TarjetaProducto />
        <TarjetaProducto />
        <TarjetaProducto />
      </div>
    </section>
  );
}
```

3. ¿Qué son los props en React? ¿Cuáles son sus características y para qué sirve? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descrito, explicando el ejemplo.

Son argumentos o parámetros que se le pasan a un componente; funcionan como los argumentos de una función, permitiendo que los componentes sean dinámicos y reutilizables. Un componente de React recibe props (en forma de un objeto) para saber qué datos debe renderizar en la pantalla. Permiten que los componentes se comuniquen entre sí y es la manera que tenemos de configurar cada componente de forma diferente.

Los props siempre se pasan de un componente padre a un componente hijo. Un hijo no puede pasarle props a su padre. Son inmutables (read only) ya que un componente hijo no puede modificar los props que recibe de un componente padre. Para cambiar un valor en base a una respuesta del usuario usamos los states, no los props. Un prop soporta cualquier tipo de dato (string, números, arreglos, objetos, otras funciones, etc).

```
test.py

// Componente hijo recibe los props)
// React agrupa todos los atributos pasados y los entrega como un objeto llamado "props"
function TarjetaUsuario(props) {
  return (
    <div className="tarjeta">
      {/* Accedemos a los valores del objeto props usando {} */}
      <h2>Nombre: {props.nombre}</h2>
      <p>Profesión: {props.profesion}</p>
      <p>Edad: {props.edad} años</p>
    </div>
  );
}

// El componente padre envía los props
export default function Aplicacion() {
  return (
    <section>
      <h1>Lista de Empleados</h1>

      {/* Reutilizamos el mismo componente con distintos props */}

      <TarjetaUsuario
        nombre="Ana Gómez"
        profesion="Desarrolladora Frontend"
        edad={28}
      />

      <TarjetaUsuario
        nombre="Carlos Ruiz"
        profesion="Diseñador UX/UI"
        edad={34}
      />

    </section>
  );
}
```

4. ¿Qué es Firebase? ¿Cuáles son sus características?

Firebase es una plataforma de desarrollo de aplicaciones móviles y web creada originalmente por una startup homónima en 2011 y adquirida por Google en 2014.

Es una plataforma en la nube que funciona como un “backend como servicio” (BaaS). Proporciona toda la infraestructura y las herramientas de servidor necesarias (como bases de datos, autenticación, almacenamiento de archivos y alojamiento) ya configuradas y listas para usar; por esto mismo permite que los desarrolladores de aplicaciones web y móviles trabajen con mayor velocidad y comodidad, ya que no tienen que programar un servidor propio desde cero.

Ofrece bases de datos NoSQL para guardar y sincronizar información entre usuarios de forma instantánea (usando Cloud Firestore o Realtime Database). Con *Firebase Authentication* nos permite implementar sistemas de inicio de sesión seguros ya sea con las credenciales “tradicionales” (email y contraseña) o con otras (Google, GitHub, Facebook, Apple, etc.).

Respaldo por la infraestructura de Google Cloud, ofrece escalabilidad automática que permite a tu aplicación crecer sin necesidad de gestionar servidores físicos o virtuales. Usa una arquitectura “serverless” que permite ejecutar código backend personalizado ante eventos específicos. Ofrece web hosting. Es multiplataforma ya ofreciendo SDKs (kits de desarrollo de software) oficiales que permiten compartir la misma base de datos entre la Web, iOS, Android, Unity y C++.